



UNIVERSITY OF TRENTO

Department of Information Engineering and Computer Science

Master's Degree in
Computer Science

FINAL DISSERTATION

MAINTAINING HITTING SETS IN TEMPORAL HYPERGRAPHS

Subtitle

Supervisor
Alberto Montresor
Co-Supervisor
Quintino Francesco Lotito

Student
Luca Prigione
242880

Academic year 2025/2026

Contents

Abstract	1
1 Introduction	2
1.1 Context	2
1.2 Problem Statement	2
2 Background and Related Work	4
3 Proposed Approach	5
3.1 Formal Problem Definition	5
3.1.1 Introduction	5
3.1.2 Objective	5
4 Experimental evaluation	6
5 Conclusions and Future Work	7
Bibliography	7
A Attachment	9

Abstract

1 Introduction

1.1 Context

The increasing availability of relational data studied in recent years has led to a growing interest in the study of data structures capable of representing complex interactions between entities. In the literature, these relationships were originally managed by means of *graphs*, which are mathematical structures used to model one-to-one relations between objects. However, many real-world scenarios find this representation too restrictive, especially when considering interactions between more than just two entities. Several real-life scenarios cannot be represented as graphs without a significant information loss. Some examples include conversations between multiple people, usually found online in forums, chat or more generally in social media, as well as joint interactions among multiple objects or entities, such as those involved in a bank transaction. In these contexts, information loss is unavoidable when restricting the representation to pairwise edges. To overcome these limitations, the concept of *hypergraph* was introduced. Hypergraphs generalize standard graphs by allowing each hyperedge to connect an arbitrary number of nodes, making them suitable for modelling higher-order interactions [1]. This type of data structure is particularly useful when the goal is to represent complex interactions without losing structural information, which traditional graphs tend to discard. Furthermore, in practical scenarios the interactions between entities are not static, but they evolve over time. New interactions are created, while others tend to disappear, resulting in a complex, dynamic system that brings a theoretically static data structure to evolve over time. This type of challenge makes the management of this temporal evolution a necessity, in order to be able to represent the data always in an accurate way. “Just to mention a few examples, temporal graph modelling and analysis of temporal properties can have applications in sociology and social network analysis [...]; security and distributed computing [...]; biology” [3]. The analysis of large dynamic structures also introduces important computational problems [3]. In particular, it is often necessary to continuously update the information of interest without being able to recalculate the entire solution from scratch at each time step. Consequently, the study of efficient techniques for the maintenance of combinatorial properties and structures in dynamic and temporal environments plays a central role.

1.2 Problem Statement

When analyzing systems characterized by collective interactions, it is often necessary to find a restricted set of elements aimed at controlling, representing, or covering the entirety of the interactions present in the system. Some of the most famous problems in theoretical computer science defined on graphs and hypergraphs focus precisely on finding a minimum subset of elements that satisfies a given property. Among the most famous examples are the *vertex cover* and the *set cover* problems, optimization problems in which the objective is to minimize, respectively, the number of edges and nodes required to cover all the remaining elements, famous for their computational complexity belonging to NP-hard problems [2]. Unlike the previous problems, in this work, the interactions are not static but evolve over time, and furthermore, only hypergraphs are considered as data structures. Consequently, the objective consists not exclusively in determining a globally optimal solution, but rather in maintaining a valid solution over time that can be efficiently updated as interactions change, avoiding the need to fully recalculate it at every modification of the system. To describe it more precisely, the problem addressed in this work consists in maintaining a set of nodes that can cover all the interactions represented by the active hyperedges of a temporal hypergraph. The solution must therefore adapt dynamically to the evolution of the structure, maintaining the coverage property. In addition to the average size of the solution, further evaluation criteria are also considered. To have a solution that is valid not only in a theoretical context but also in a practical one, consideration is given both to the execution time required to update the solution as interactions change, and to

the total number of distinct nodes that become part of the selected set during the evolution of the hypergraph. All this is to ensure that the solution allows for the fewest total modifications and that it is effectively usable even in real-world contexts, where the speed of updating the solution is also required, in addition to its correctness.

2 Background and Related Work

3 Proposed Approach

3.1 Formal Problem Definition

3.1.1 Introduction

We consider an online variant of the Temporal Hitting Set problem under a sliding window model. To this end, we model the input as a temporal hypergraph $H = (V, E)$, where each hyperedge (S, t) consist of a subset of vertices $S \subseteq V$ and a discrete time step $t \in \mathbb{T}$.

Given a time window of size ϵ , we define the set of active hyperedges at time t as:

$$W(t) = \{(S, t') \in E \mid t - \epsilon \leq t' \leq t\}.$$

At each time step t , the algorithm maintains a hitting set $X(t) \subseteq V$ such that:

$$\forall (S, t') \in W(t), \quad X(t) \cap S \neq \emptyset.$$

3.1.2 Objective

The goal is to maintain a feasible hitting set over time while optimizing the following criteria:

- **Solution size:** minimize $|X(t)|$ at each time step;
- **Update cost:** minimize the number of new nodes taken in consideration when updating the hitting set from $X(t - 1)$ to $X(t)$;
- **Computational efficiency:** minimize the time required to update the solution as the sliding window evolves.

Overall, the objective is to maintain a small and stable hitting set while efficiently handling the insertion and expiration of hyperedges induced by the sliding window.

4 Experimental evaluation

5 Conclusions and Future Work

Bibliography

- [1] Alessia Antelmi, Gennaro Cordasco, Mirko Polato, Vittorio Scarano, Carmine Spagnuolo, and Dingqi Yang. A survey on hypergraph representation learning. *ACM Computing Surveys*, 56(1):1–38, 6 2023.
- [2] Fernando C. Gomes, Cláudio N. Meneses, Panos M. Pardalos, and Gerardo Valdisio R. Viana. Experimental analysis of approximation algorithms for the vertex cover and set covering problems. *Computers & Operations Research*, 33(12):3520–3534, 2006. Part Special Issue: Recent Algorithmic Advances for Arc Routing Problems.
- [3] Quintino Francesco Lotito and Alberto Montresor. Efficient algorithms to mine maximal span-trusses from temporal graphs. In *Proceedings of the 16th International Workshop on Mining and Learning with Graphs (MLG '20)*, 2020.

Appendix A Attachment

Acknowledgements

Thanks to ...